Chapter 1 – Organisation

1 Organisation

2 Contenu de l'enseignement

Equipe pédagogique

Silviu Maniu (CM, G4)	Professeur UGA, LIG Silviu.Maniu@univ-grenoble-alpes.fr
Mario Cortes-Cornax (CM, G3)	Maître de Conférences UGA, LIG Mario.Cortes-Cornax@univ-grenoble-alpes.fr
Michael Magi (G1)	Ingénieur développement d'applications, UFR IM^{2}AG Michael.Magi@univ-grenoble-alpes.fr
Catherine Vigouroux (G2)	Maître de Conférences UGA, Verimag Catherine.Vigouroux@univ-grenoble-alpes.fr

Volumes horaires

- Cours Magistral (CM) : 16.5
- Travaux Dirigés - Pratiques (TD-TP) : 33
- Heures encadrées : 49.5
- Travail personnel estimé : 33

Evaluation

Quick (23/10/2026)	qck
Projet (27/11/2026)	prj
Contrôle continu	cc = (qck + prj) / 2
Examen (2 heures)	ex
Note CEBD	Max(ex, 0.55 * ex + 0.45 * cc)

Absence (non justifiée) à une des épreuves du contrôle continu
==> ABI à cette épreuve ==> défaillance à l'UE

Apnée (09/10/2026) - entrainement personnel, pas notée, évaluée automatiquement par Caseine

“Règles du jeu”

- CM - pratique
- Slides en anglais
- Contenu du cours / TPs / Rendus ==> Caseine (`https://moodle.caseine.org/course/view.php?id=181`)

Chapter 1 – Organisation

1 Organisation

2 Contenu de l'enseignement

Pré requis pour cette UE

- Algorithmique et de programmation.
- Logique : principes de base.
- Théorie des ensembles : ensembles, opérateurs sur les ensembles ($\in$, $\notin$, $\subset$, $\not\subset$, $\cap$, $\cup$, $-$, X, etc).
- Modèle relationnel de données : relation, attribut, domaine
- SQL (interrogation et création) : voir cours L2

Plan

- Partie I: Bases de données relationnelles
  1. Introduction
  2. Le modèle relationnel de données
  3. UML et transformation relationnel
  4. Algèbre Relationnelle
  5. SQL (DML, DDL, Triggers)
- Partie II: Normalisation et projet
  1. Normalisation
  2. Index et execution de requêtes (optimisation)
  3. Projet pratique

Compétences visées

L'objectif est d'acquérir les connaissances minimales sur les bases de données à tout informaticien :

Maîtriser

- la compréhension d'un schéma relationnel
- l'expression de requêtes d'interrogation en algèbre relationnelle et en SQL
- la traduction en SQL d'une requête exprimée dans l'algèbre relationnelle

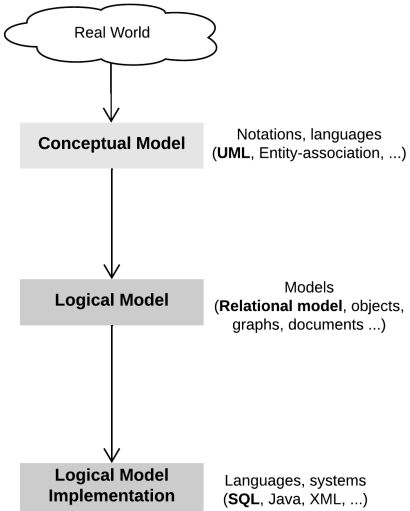
Savoir

- évaluer et améliorer la qualité d'un schéma de base de données
- modéliser une base de données à l'aide d'UML
- partir d'une conception pour arriver jusqu'à l'implementation d'une base de données

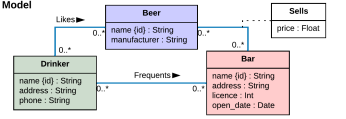
Chapter 2 – Introduction

3 Introduction

Data design process



Conceptual Model



Logical Model

Beers (beer_name, beer_manufacturer)

Drinkers (drinker_name, drinker_address, drinker_phone)

Bars (bar_name, bar_address, bar_licence, bar_open_date)

Likes (drinker_name, beer_name)

Likes[drinker_name] ⊆ Drinkers[drinker_name]

Likes[beer_name] ⊆ Beers[beer_name]

Frequents (drinker_name, bar_name)

Frequents[drinker_name] ⊆ Drinkers[drinker_name]

Frequents[bar_name] ⊆ Bars[bar_name]

Sells (bar_name, beer_name, sells_price)

sells_price > 0

Sells[beer_name] ⊆ Beers(beer_name)

Sells[bar_name] ⊆ Bars[bar_name]

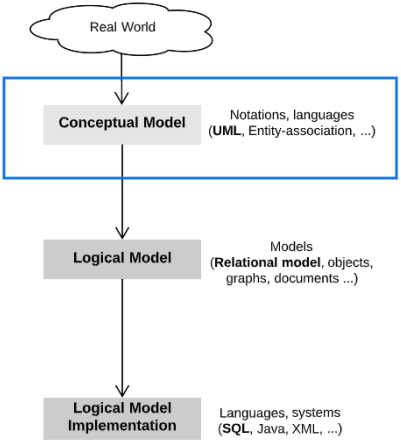
Logical Model Implementation

```
-- SQLite Syntax
CREATE TABLE Beers (
  beer_name TEXT NOT NULL,
  -- NOT NULL not necessary for PK attributes in some systems such as ORACLE
  beer_manufacturer TEXT,
  CONSTRAINT pk_beers_c0 PRIMARY KEY (beer_name)
);

CREATE TABLE Bars (
      bar_name TEXT NOT NULL,
      bar_address TEXT,
      bar_licence DATE,
      bar_open_date DATE,
      CONSTRAINT pk_bars_c0 PRIMARY KEY (bar_name)
);

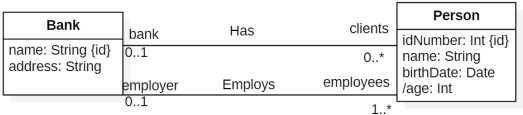
CREATE TABLE Sells(
  bar_name TEXT NOT NULL,
  beer_name TEXT NOT NULL,
  sells_price REAL NOT NULL,
  CONSTRAINT pk_sells_c0 PRIMARY KEY (bar_name,beer_name),
  CONSTRAINT fk_sells_c1 FOREIGN KEY (bar_name) REFERENCES Bars(bar_name),
  CONSTRAINT fk_sells_c2 FOREIGN KEY (beer_name) REFERENCES Beers(beer_name),
  CONSTRAINT ck_sells_c3 CHECK (sells_price > 0)
);
```

Data design process



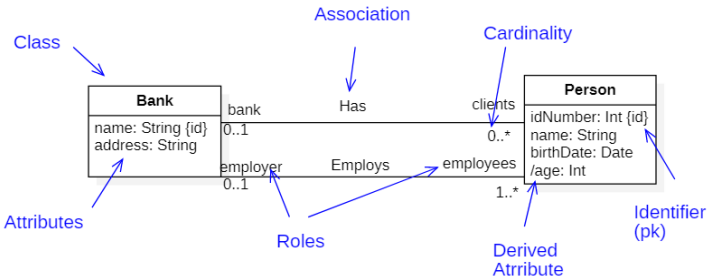
The Main UML Class Diagram* Elements

*UML Adaptation for DBs

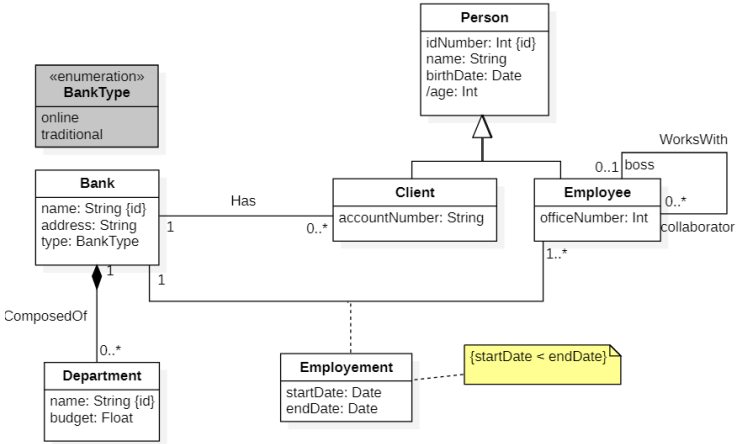


The Main UML Class Diagram* Elements

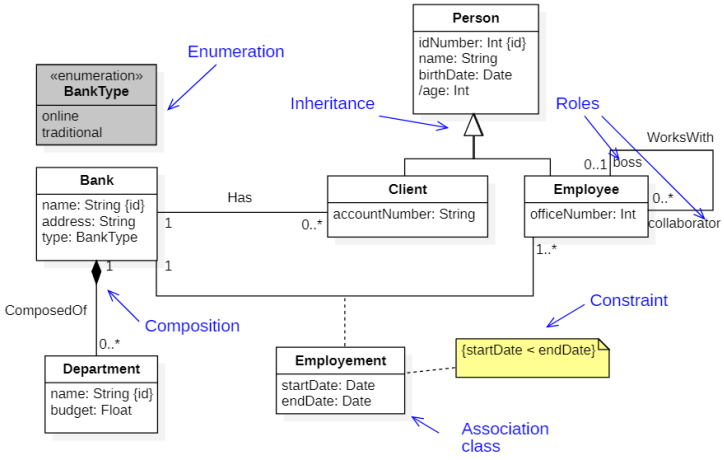
*UML Adaptation for DBs



Extended UML class diagram



Extended UML class diagram



Relational Model vs UML Class Diagram (DB)

The relational model and UML class diagram have some obvious correspondences:

Relational Model	*UML Class Diagram*
Attributes	Attributes
Relation schema	Classes and Associations
Tuple	Object value

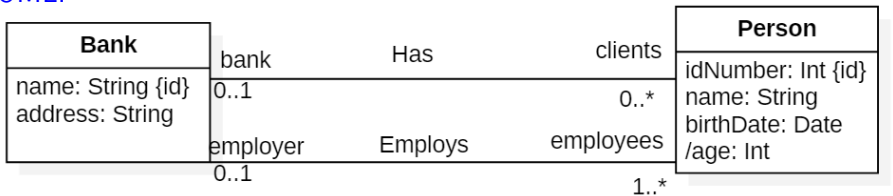
Some major differences:

1. Relations are used to model both classes and associations
2. Relational model does not support object-oriented notions (such as subclasses, inheritance)

Example UML to Relational Model*

*This transformation will be treated from next week. Have a look to L2 course for an introduction in french.

UML:



Relational model:

Banks(<u>name</u>, address)

Persons_base(<u>idNumber</u>, name, birthDate, bank, employer)

Persons_base[bank] $\subseteq$ Banks[name]

Persons_base[employer] $\subseteq$ Banks[name]

(View) Persons(idNumber, name, birthDate, bank, employer, age)

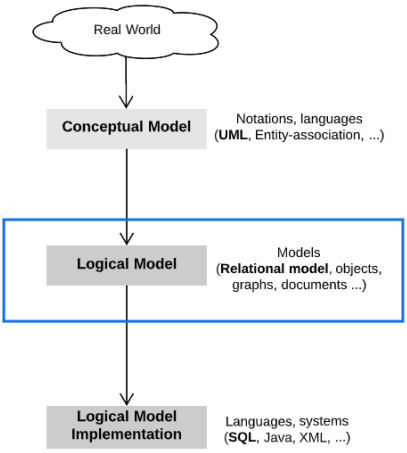
Chapter 3 – Relational Data Model

4 Preliminaries

5 Definitions and Notations

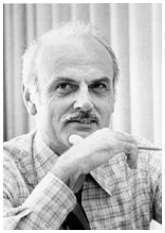
6 Specification of Relations: Example

Data design process



The Relational Data Model...

- was first introduced by Ted Codd in 1970,
- is attractive because of its simplicity and its mathematical foundations,
- uses the concept of mathematical relations as a means of modeling information,
- cannot be ignored because of its popularity.



IBM

SQL

source : `https://en.wikipedia.org/wiki/Edgar_F._Codd`

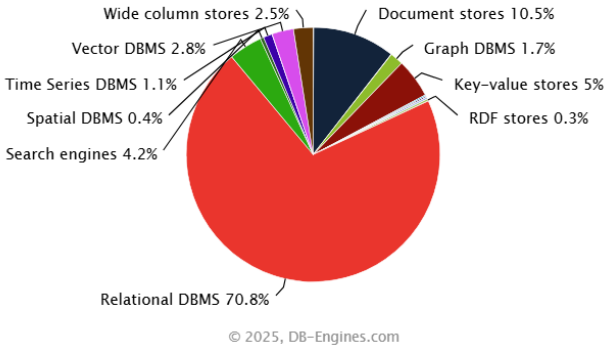
Larry Ellison (1944 -) - Oracle Co-founder



Fundamental contributions for database management systems (DBMS). Co-founder of Oracle

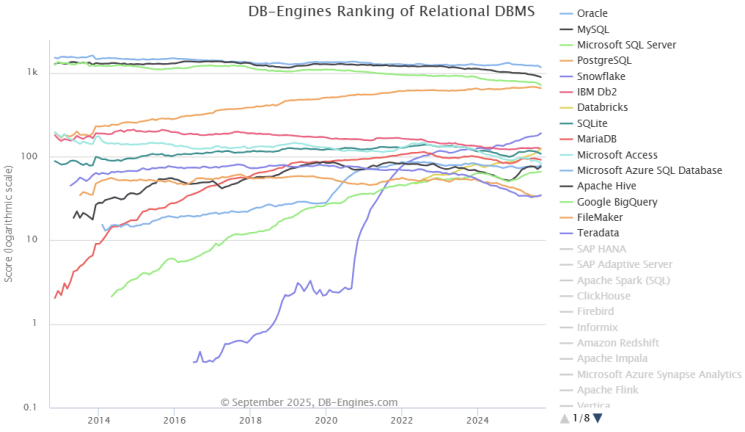
source : `https://en.wikipedia.org/wiki/Larry_Ellison`

Database Model Popularity



source : `https://db-engines.com/en/ranking_categories`

DB-Engines Ranking - Trend Popularity



source : `https://db-engines.com/en/ranking_trend`

How popularity is measured in DB-Engines Ranking ?

- Number of mentions of the system on websites
- General interest in the system (frequency of searches in Google Trends)
- Frequency of technical discussions about the system (Stack Overflow and DBA Stack Exchange)
- Number of job offers, in which the system is mentioned (Indeed and Simply Hired)
- Number of profiles in professional networks, in which the system is mentioned (LinkedIn and Upwork)
- Relevance in social networks (number of Twitter tweets in which the system is mentioned)

How the Relational Model Represents Information?

Facts about a police registry:

- El Profesor, Tokio, Nairobi, Denver and Rio criminal's pseudonyms on the police registry,
- Tokio's real name is Silene, Nairobi's one is Agata, etc.
- Nairobi's speciality is Cunterfeiter, Tokio and Berlin are Robbers, etc.
- Rio's birthyear is 1995, Nairobi's one is 1987, etc.

pseudo	**name**	**speciality**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Operations on Relations

pseudo	name	speciality	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties
- **Selection** : Give criminals that born before 1985
- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	speciality	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985

- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	speciality	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio?
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	speciality	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio? $\longrightarrow$ {<Silene>}
- **Aggregation**: how many criminals are Robbers?

Operations on Relations

pseudo	name	speciality	birthyear
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	Counterfeiter	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Querying relations

- **Projection** : Give the criminal's specialties $\longrightarrow$ {<Band Leader>, <Robber>, <Counterfeiter>, <Burglar>, <Hacker>}
- **Selection** : Give criminals that born before 1985 $\longrightarrow$ {<El Profesor, Sergio, Band Leader, 1983>, <Berlin, Andres, Robber, 1979>}
- **Selection + projection**: what is the real name of Tokio? $\longrightarrow$ {<Silene>}
- **Aggregation**: how many criminals are Robbers? $\longrightarrow$ {<2>}

Updating relations

- Nairobi's speciality has been changed to Robber

pseudo	**name**	**speciality**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	*Robber*	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979

Updating relations

- Nairobi's speciality has been changed to Robber
- Moscu is now identified in the police's registry. He's name is Agustin, he is a Burglar and his birthyear is 1968

pseudo	**name**	**speciality**	**birthyear**
El Profesor	Sergio	Band Leader	1983
Tokio	Silene	Robber	1989
Nairobi	Agata	*Robber*	1987
Denver	Daniel	Burglar	1987
Rio	Anibal	Hacker	1995
Berlin	Andres	Robber	1979
Moscu	*Agustin*	*Burglar*	*1968*

Adding new kind of information

Each criminal is involved in a crime, where a number, type and location have to be stored.

Adding new kind of information

Each criminal is involved in a crime, where a number, type and location have to be stored.

Two solutions..... add column(s)
add relation(s)

Add columns (♯ 1):

pseudo	name	speciality	birthyear	crimeId	type	location
El Profesor	Sergio	Band Leader	1983	2017052	Heist	FNMT*
Tokio	Silene	Robber	1989	2017052	Heist	FNMT
Nairobi	Agata	Counterfeiter	1987	2017052	Heist	FNMT
Denver	Daniel	Burglar	1987	2017052	Heist	FNMT
Rio	Anibal	Hacker	1995	2017052	Heist	FNMT
Berlin	Andres	Robber	1979	2017052	Heist	FNMT

* Fábrica Nacional de Moneda y Timbre

Problem: Data redundancy...

The "Heist is done in the FNMT" is said as many times as there are criminals involved in the crime.

Add a relation (♯ 2):

pseudo	name	speciality	birthyear	crimeId
El Profesor	Sergio	Band Leader	1983	2017052
Tokio	Silene	Robber	1989	2017052
Nairobi	Agata	Counterfeiter	1987	2017052
Denver	Daniel	Burglar	1987	2017052
Rio	Anibal	Hacker	1995	2017052
Berlin	Andres	Robber	1979	2017052

crimeId	type	location
2017052	Heist	FNMT

Problem: Queries are a bit more complex...

"Find the location of the crime in which Denver was involved"

Chapter 3 – Relational Data Model

4 Preliminaries

5 Definitions and Notations

6 Specification of Relations: Example

Set: Definition and Some Operations

- A set is a collection of elements (enclosed by braces) pairwise different and usually related to a particular domain.
  $A = \{\text{'Robber', 'Hacker', 'Burglar'}\}$
  $F = \{p \in \text{Persons: Gender}(p) = \text{'female'}\}$
  $G = \{1, 5, 6, 3\}$

A and G are given by extension, while F is so by intension.

Set: Definition and Some Operations

- A set is a collection of elements (enclosed by braces) pairwise different and usually related to a particular domain.
  $A = \{\text{'Robber', 'Hacker', 'Burglar'}\}$
  $F = \{p \in \text{Persons: Gender}(p) = \text{'female'}\}$
  $G = \{1, 5, 6, 3\}$
- Membership:
  $3 \in \{1, 3, 5, 6\}$
  $8 \notin \{1, 3, 5, 6\}$

A and G are given by extension, while F is so by intension.

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$
- Minus (asymetrical):
  $\{1, 3, 5, 6\} - \{10, 5, 3, 9\} = \{1, 6\}$
  $\{10, 5, 3, 9\} - \{1, 3, 5, 6\} = \{10, 9\}$

- Cartesian product of sets (denoted X):
  {1, 5, 6, 3} X *{'Robber', 'Hacker', 'Burglar'}*
  { <1,'Robber'>, <1,'Hacker'>, <1,'Burglar'>,
  <3,'Robber'>, <3,'Hacker'>, <3,'Burglar'>,
  <5,'Robber'>, <5,'Hacker'>, <5,'Burglar'>,
  <6,'Robber'>, <6,'Hacker'>, <6,'Burglar'> }
- Intersection: $\{1, 3, 5, 6\} \cap \{10, 5, 3, 9\} = \{5, 3\}$
- Union:
  $\{1, 3, 5, 6\} \cup \{10, 5, 3, 9\} = \{1, 5, 3, 6, 10, 9\}$
- Minus (asymetrical):
  $\{1, 3, 5, 6\} - \{10, 5, 3, 9\} = \{1, 6\}$
  $\{10, 5, 3, 9\} - \{1, 3, 5, 6\} = \{10, 9\}$
- Inclusion:
  $\{\ \} \subseteq \{10, 5, 3, 9\}$ ({ } is also denoted $\emptyset$)
  $\{9, 10\} \subseteq \{10, 5, 3, 9\}$
  $\{9, 10\} \subset \{10, 5, 3, 9\}$
  $\{9, 10, 3, 5\ \} \not\subset \{10, 5, 3, 9\}$
  $\{1, 9, 10, 3, 5, 7\} \not\subseteq \{10, 5, 3, 9\}$

Question

Cartesian product of sets:

$\{1, 3\}$ X $\{2, 1\} =$

A. $\{ <1, 2>, <1,1>, <3,2>, <3,1>\}$
B. $\{ <1, 2>, <3,1>\}$
C. $\{ <1, 3, 2, 1>\}$

Question

Cartesian product of sets:

$\{1, 3\}$ X $\{2, 1\} = \{ <1, 2>, <1,1>, <3,2>, <3,1>\}$

A. $\{ <1, 2>, <1,1>, <3,2>, <3,1>\}$
B. $\{ <1, 2>, <3,1>\}$
C. $\{ <1, 3, 2, 1>\}$

Question

Intersection: $\{1, 2, 5, 7\} \cap \{5, 3, 1\} =$

A. $\{1, 2, 5, 7, 3\}$

B. $\{2, 7\}$

C. $\{1, 5\}$

Question

Intersection: $\{1, 2, 5, 7\} \cap \{5, 3, 1\} = \{1, 5\}$

A. $\{1, 2, 5, 7, 3\}$

B. $\{2, 7\}$

C. $\{1, 5\}$

Question

Union: $\{2, 1, 4, 7\} \cup \{1, 5, 3\} =$

A. $\{2, 4, 7\}$

B. $\{1\}$

C. $\{2, 1, 4, 7, 5, 3\}$

Question

Union: $\{2, 1, 4, 7\} \cup \{1, 5, 3\} = \{2, 1, 4, 7, 5, 3\}$

A. $\{2, 4, 7\}$

B. $\{1\}$

C. $\{2, 1, 4, 7, 5, 3\}$

Question

Minus (asymetrical):

$\{1, 2, 5, 7\} - \{5, 3, 1\} =$

A. $\{2, 7, 3\}$
B. $\{2, 7\}$
C. $\{3\}$

Question

Minus (asymetrical):

$\{1, 2, 5, 7\} - \{5, 3, 1\} = \{2, 7\}$

A. $\{2, 7, 3\}$
B. $\{2, 7\}$
C. $\{3\}$

Question

Inclusion:

$\{2, 1, 4, 7\} \subset \{7, 2, 1, 4\}$ (True or False?)
$\{4, 2\} \not\subseteq \{7, 2, 1, 4\}$ (True or False?)

A. True and True
B. False and True
C. False et False

Question

Inclusion:

$\{2, 1, 4, 7\} \subset \{7, 2, 1, 4\}$ (True or False?) F
$\{4, 2\} \not\subseteq \{7, 2, 1, 4\}$ (True or False?) F

A. True and True
B. False and True
C. False et False

Domain, Relation, Attribute, Schema

- A **domain** is a set of atomic values (strings, s,...).
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$, $\{$*integers* $> 1900\}$

Domain, Relation, Attribute, Schema

- A **domain** is a set of atomic values (strings, s,...).
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$, $\{$*integers* $> 1900\}$
- A **relation** (table) is a subset of the cartesian product of a set of domains.
  $\{<$*'El Profesor',1983*$>, <$*'Tokio',1989*$>, <$*'Nairobi',1987*$>, <$*'Denver',1987*$>,$
  $<$*'Rio',1979*$>\}$
  $\subseteq$
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$ *X* $\{$*integers* $> 1900\}$

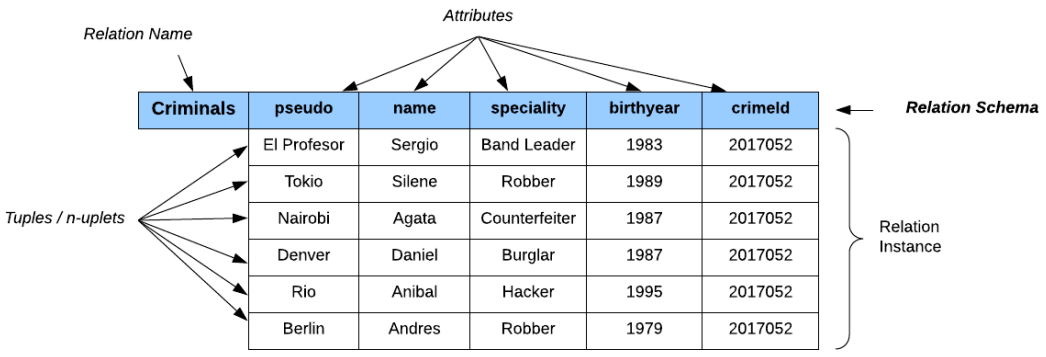
Domain, Relation, Attribute, Schema

- A **domain** is a set of atomic values (strings, s,...).
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$, $\{$*integers* $> 1900\}$
- A **relation** (table) is a subset of the cartesian product of a set of domains.
  $\{<$*'El Profesor',1983*$>, <$*'Tokio',1989*$>, <$*'Nairobi',1987*$>, <$*'Denver',1987*$>, <$*'Rio',1979*$>\}$
  $\subseteq$
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$ *X* $\{$*integers* $> 1900\}$
- An **attribute** specifies a name for a domain in a relation.
  dom(birthyear) $= \{$ *integer* $> 1900\}$

Domain, Relation, Attribute, Schema

- A **domain** is a set of atomic values (strings, s,...).
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$, $\{$*integers* $> 1900\}$
- A **relation** (table) is a subset of the cartesian product of a set of domains.
  $\{<$*'El Profesor',1983*$>, <$*'Tokio',1989*$>, <$*'Nairobi',1987*$>, <$*'Denver',1987*$>, <$*'Rio',1979*$>\}$
  $\subseteq$
  $\{$*'El Profesor', 'Tokio', 'Nairobi', 'Denver', 'Rio'*$\}$ *X* $\{$*integers* $> 1900\}$
- An **attribute** specifies a name for a domain in a relation.
  dom(birthyear) $= \{$ *integer* $> 1900\}$
- A relation **schema** is specified by its name and a set of attributes.
  Criminals (pseudo, name, speciality, birthyear, crimeId)

Example:



Criminals	pseudo	name	speciality	birthyear	crimeId
	El Profesor	Sergio	Band Leader	1983	2017052
	Tokio	Silene	Robber	1989	2017052
	Nairobi	Agata	Counterfeiter	1987	2017052
	Denver	Daniel	Burglar	1987	2017052
	Rio	Anibal	Hacker	1995	2017052
	Berlin	Andres	Robber	1979	2017052

Attributes and tuples have no order

Interpretation

An interpretation (spécification in French) of a relation schema as a predicate:

Criminals (pseudo, name, speciality, birthyear, crimeId)

/ <p, na, s, b, cn> ∈ *Criminals* ⟺ *the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, birthyear b and has been involved in crime crimeId cn. */*

This is useful to understand a relation schema.

Relational Constraints

- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.

Relational Constraints

- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.
- *Key constraint:*
  $\underline{X}$ specifies an uniqueness constraint so that tuples of the relation are pairwise different for X (X set of attributes).

Relational Constraints

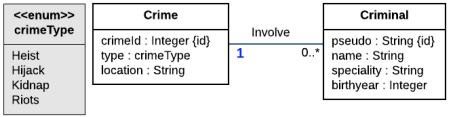
- *Domain constraint:*
  domain(A)=T specifies that the values of A must be from the type T.
- *Key constraint:*
  $\underline{X}$ specifies an uniqueness constraint so that tuples of the relation are pairwise different for X (X set of attributes).
- *Referential integrity constraint:*
  R on attributes X refers to S on attributes Y (Notation: R[X] $\subseteq$ S[Y]). All tuples in R restricted to X must have a corresponding tuple in S that matches on Y.

Chapter 3 – Relational Data Model

4 Preliminaries

5 Definitions and Notations

6 Specification of Relations: Example



Criminals (pseudo, name, speciality, birthyear, crimeId)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, a birthyear b and she(he) has been involved in the crime crimeId cn. */*

Crimes(crimeId, type, location)

/ <n, t, l> ∈ Crimes ⟺ the crime identified by a crimeId n, has a type t (Heist, Hijack, Riots, Kidnap) and a location l */*

Domain constraints:

domain (pseudo) = domain (name) = domain (speciality) = domain (location) = strings

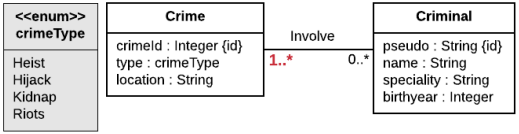
domain (birthyear) = number > 1900

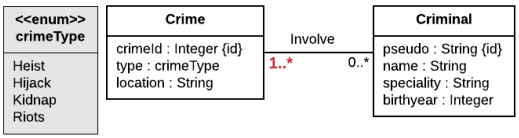
domain (crimeId) = number > 0

domain (type) = {'Heist', 'Hijack', 'Riots', 'Kidnap'}

Referential integrity constraint:

Criminals[crimeId] ⊆ Crimes[crimeId]


<<enum>>
crimeType
Heist
Hijack
Kidnap
Riots
Crime
crimeId : Integer {id}
type : crimeType
location : String
Involve
1..*
0..*
Criminal
pseudo : String {id}
name : String
speciality : String
birthyear : Integer



Criminals (pseudo, name, speciality, birthyear)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s and a birthyear b. */*

Crimes(crimeId, type, location)

...

Involvements(crimeId, pseudo)

/ <ci, cp> ∈ Involvements ⟺ the crime identified by an id ci involves the criminal identified by the pseudo cp */*

Domain constraints:

...

Referential integrity constraint:

Involvements[crimeId] ⊆ Crimes[crimeId]

Involvements[pseudo] = Criminals[pseudo]

Chapter 4 – UML to REL

7 Introduction

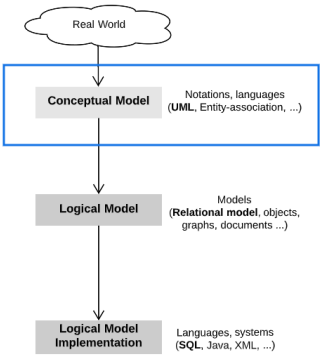
8 Mapping classes

9 Mapping Associations and Association classes

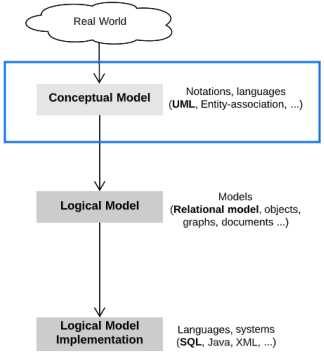
10 Mapping class Hierarchy

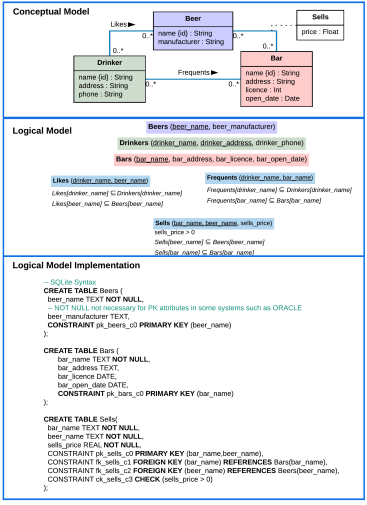
11 Concluding Remarks

Data design process



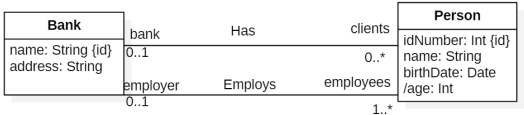
Data design process





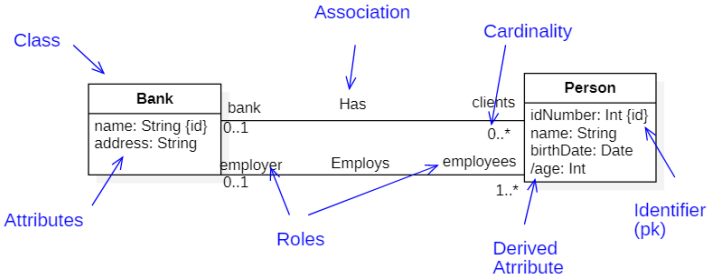
The Main UML Class Diagram* Elements

*UML Adaptation for DBs



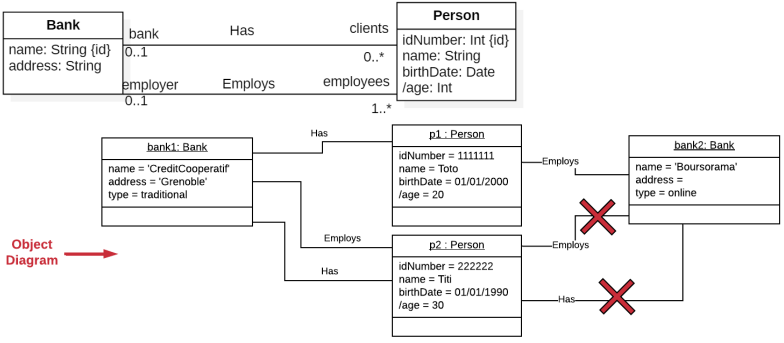
The Main UML Class Diagram* Elements

*UML Adaptation for DBs

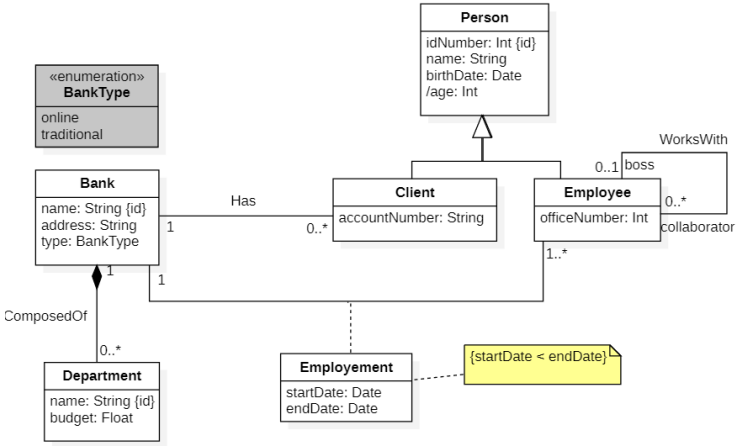


Object Diagram

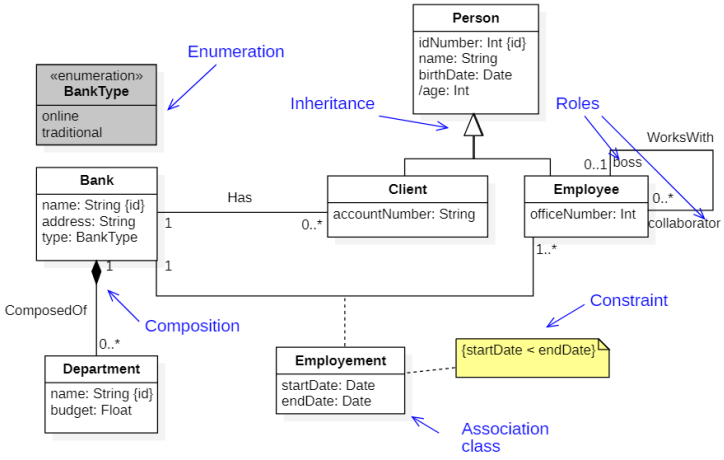
*UML Adaptation for DBs



Extended UML class diagram



Extended UML class diagram



Relational Model vs UML Class Diagram (DB)

The relational model and UML class diagram have some obvious correspondences:

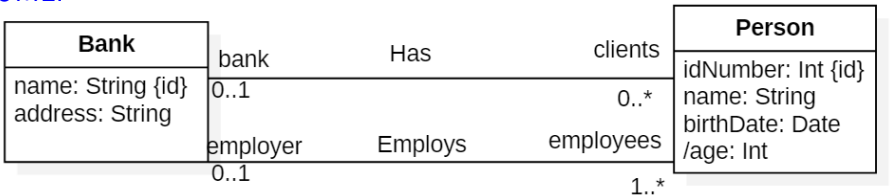
Relational Model	*UML Class Diagram*
Attributes	Attributes
Relation schema	Classes and Associations
Tuple	Object value

Some major differences:

1. Relations are used to model both classes and associations
2. Relational model does not support object-oriented notions (such as subclasses, inheritance)

Example UML to Relational Model (1)

UML:



Relational model:

Banks(<u>name</u>, address, type)

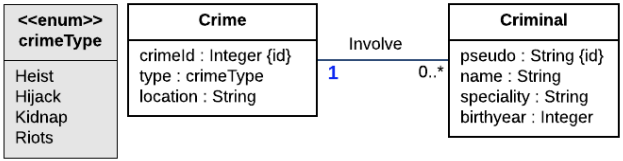
Persons_base(<u>idNumber</u>, name, birthYear, bank, employer)

Persons_base[bank] $\subseteq$ Banks[name]

Persons_base[employer] $\subseteq$ Banks[name]

(View) Persons(idNumber, name, birthYear, bank, employer, age)

Example UML to Relational Model (2)



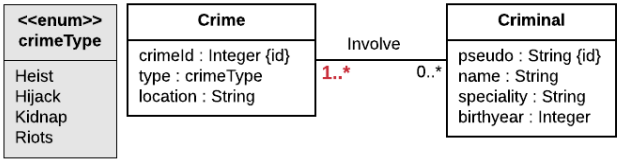
Criminals (pseudo, name, speciality, birthyear, crimeId)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s, a birthyear b and she(he) has been involved in the crime crimeId cn. */*

Crimes(crimeId, type, location)

/ <n, t, l> ∈ Crimes ⟺ the crime identified by a crimeId n, has a type t (Heist, Hijack, Riots, Kidnap) and a location l */*

Example UML to Relational Model (3)



Criminals (pseudo, name, speciality, birthyear)

/ <p, na, s, b, cn> ∈ Criminals ⟺ the criminal identified by her(his) pseudo p has a real name na. She(he) has as speciality s and a birthyear b. */*

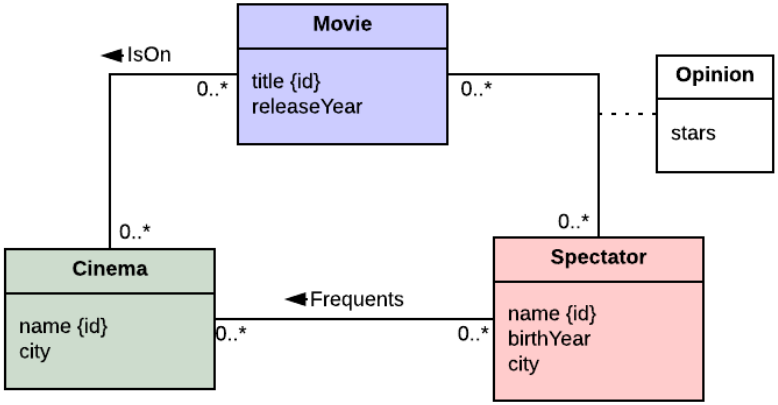
Crimes(crimeId, type, location)

...

Involvements(crimeId, pseudo)

/ <ci, cp> ∈ Involvements ⟺ the crime identified by a crime Id ci involves criminal identified by pseudo cp */*

Example UML to Relational Model (4)



Example UML to Relational Model (4)

Toy Database Relational Model

- *Movies (title, releaseYear)*
  $< t, y > \in Movies \iff$ {the movie is identified by the title t and has a release year y}
- *Cinemas (name, city)*
  $< n, c > \in Cinemas \iff$ {the cinema identified by its name n is located in the city c}
- *Spectators (name, birthYear, city)*
  $< n, b, c > \in Spectators \iff$ ...
  Domains need to be detailed.
- *Frequents (spectator, cinema)*
  $< s, c > \in Frequents \iff$ ...
  Frequents[spectator] $\subseteq$ Spectators[name]
  Frequents[cinema] $\subseteq$ Cinemas[name]
- *IsOn (movie, cinema)*
  $< m, c > \in IsOn \iff$ ...
  IsOn[movie] $\subseteq$ Movies[title]
  IsOn[cinema] $\subseteq$ Cinemas[name]
- *Opinions (spectator, movie, stars)*
  $< sp, m, st > \in Opinions \iff$ ...
  domain(stars) = {0, 1, 2, 3, 4, 5}
  Opinions[spectator] $\subseteq$ Spectators[name]
  Opinions[movie] $\subseteq$ Movies[title]

UML to Relational Mapping: Aims

Implementing a DBMS-based application needs to transform conceptual design into relational model.
Main objectives are:

- Represent all information contained in a class diagram (classes, associations and constraints)
- Avoid redondancy (produce a good-quality[1] relational schema)
- Provide a relational schema as simple as possible (avoid too many relations)

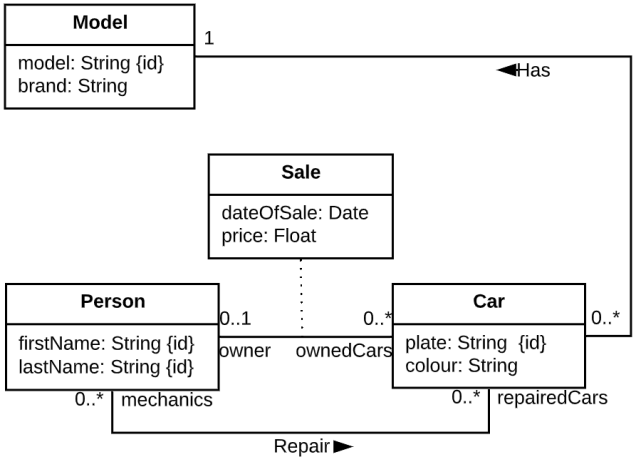
[1] 3NF schema (see later)

The Main UML Elements : Exercice

About transactions:

- A garage models the fact that a car is sold by a person (the owner) to another one.
- The brand and model of the car are *permanent* and a plate and the colour may vary.
- When a car is sold, it is so in a certain stage captured by a timestamped snapshot (e.g. the date of sale and the price is captured).
- In the garage, a car can be repaird by several mechanics, and a mechanic can repair several cars.

First UML class diagram



Chapter 4 – UML to REL

7 Introduction

8 Mapping classes

9 Mapping Associations and Association classes

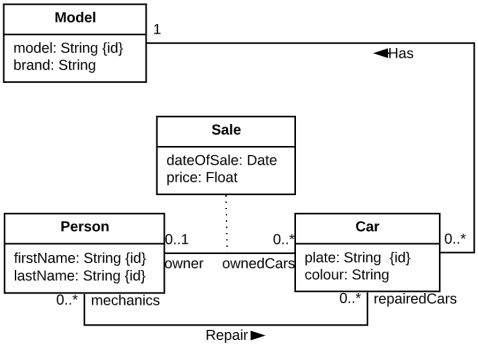
10 Mapping class Hierarchy

11 Concluding Remarks

Mapping classes

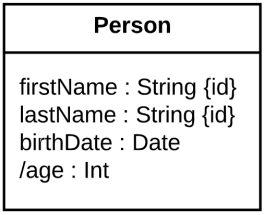
For classes that are neither abstract (nor virtual), nor association class:

- One relation for each class
- Relation attributes are class attributes
- Id elements are part of a composed key



Models(<u>model</u>, brand, ...
Cars(<u>plate</u>, colour, ...
Persons(<u>first_name, last_name</u>, ...

Classes with derived attributes




```
/* A relation to store regular attributes: */
   Persons_base (first_name, last_name, birth_date)
/* Plus a view to capture derived attributes:  */
   CREATE VIEW Persons (first_name, last_name, birth_date, age) AS
      SELECT first_name, last_name, birth_date,
         CAST(STRFTIME('%Y.%m%d', 'now') - STRFTIME('%Y.%m%d', dateNaisSp) AS INT)
      FROM Person_base;
```

Chapter 4 – UML to REL

7 Introduction

8 Mapping classes

9 Mapping Associations and Association classes

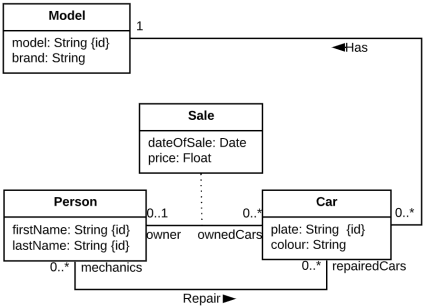
10 Mapping class Hierarchy

11 Concluding Remarks

Mapping Associations (1 to Many)

For associations 1 to Many (*):

- Add attributes in the * class referencing the key of the 1 role class
- As cardinality is 1, the new attribute (e.g., fk) cannot be null

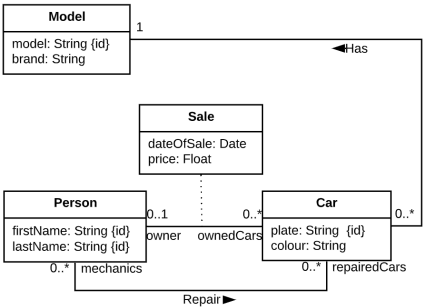


Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, **model**)
Persons (<u>first_name, last_name</u>)
...
Cars[model] $\subseteq$ Models[model]
model not null in Cars

Mapping Associations (0..1 to Many) Opt. 1 - Accept NULL values

For associations 0..1 to Many (*), first option :

- Add attributes in the * class referencing the key of the 0..1 role class (admit NULL values)



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model, **<u>owner_first_name, owner_last_name</u>, date_of_sale, price**)
Persons (<u>first_name, last_name</u>)
...

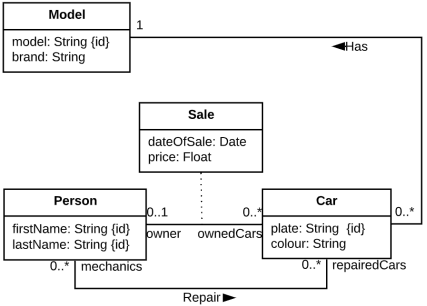
Cars[owner_first_name, owner_last_name] $\subseteq$ Persons[first_name, last_name]

owner_first_name, owner_last_name, date_of_sale, price **can be NULL**

Mapping Associations (0..1 to Many) Opt. 2 - Do not accept NULL values

For associations 0..1 to Many (*) second option :

- A new relation where the key of the 0..* role class is the key of the created relation.



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model)
Sales(owner_first_name, owner_last_name, <u>plate</u>, date_of_sale, price)
Persons (<u>first_name, last_name</u>)
...

$$Sales[plate] \subseteq Cars[plate]$$

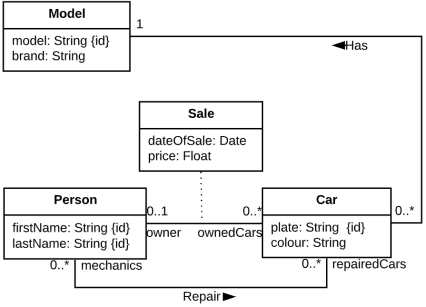
$$Sales[owner_first_name, owner_last_name] \subseteq Persons[first_name, last_name];$$

owner_first_name, owner_last_name, plate, date_of_sale, price **cannot be NULL**;

Mapping Associations (Many to Many)

For associations Many to Many:

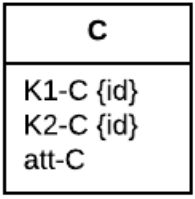
- A new relation represents the many to many association
- The key of the new relation is composed of attributes that reference the keys of the related relations



Models (<u>model</u>, brand)
Cars (<u>plate</u>, colour, model)
Sales (owner_first_name, owner_last_name, <u>plate</u>, date_of_sale, price)
Persons (<u>first_name, last_name</u>)
Reparations(<u>mech_first_name, mech_last_name, plate</u>)

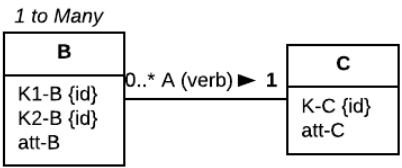
Reparations[mech_first_name, mech_last_name] $\subseteq$ Persons[first_name, last_name];
Reparations[plate] $\subseteq$ Cars[plate];

Question 1 : UML class to Relational



A. C(K1-C, K2-C, att-C)
B. C(K1-C, K2-C, att-C)
C. C(K1-C, K2-C, att-C)

Question 2 : UML 1 to Many association to Relational (class B)

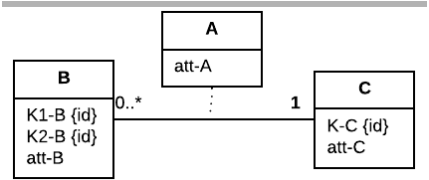


A. B(K1-B, K2-B, att-B)

B. B(K1-B, K2-B, att-B, K-C)

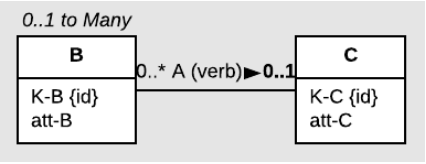
C. B(K1-B, K2-B, K-C, att-B)

Question 3 : UML Association class in 1 to Many association to Relational (class A and B)



A. B(K1-B, K2-B, att-B, K-C, att-A)
B. B(K1-B, K2-B, att-B, K-C); A(att-A)
C. B(K1-B, K2-B, att-B); A(K1-B, K2-B, K-C, att-A)

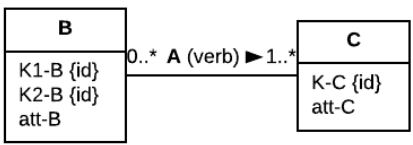
Question 4 : UML 1..0 to Many Association to Relational (class B and Association A)



A. B(<u>K-B</u>, att-B); A(<u>K-B</u>, K-C)
B. B(<u>K-B</u>, att-B); A(K-B, <u>K-C</u>)
C. B(<u>K-B</u>, att-B, K-C)

Question 5 : UML Many to Many Association to Relational

Many to Many



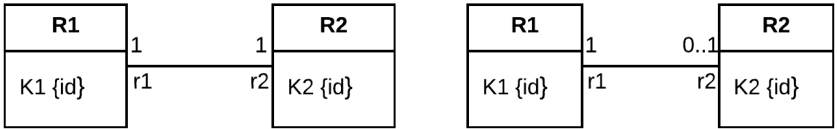
A. B(K1-B, K2-B, att-B, K-C, att-C)
B. B(K1-B, K2-B, att-B); A(K1-B, K2-B,K-C); C(K-C, att-C)
C. B(K1-B, K2-B, att-B); A(K1-B, K2-B, K-C); C(K-C, att-C)

Reminder of Rules with Many to Many Asssocations

A new relation R is created (related to the association itself):

- K1 candidate key in R1, K2 candidate key in R2
- Attributes in R: K1 $\cup$ K2
- Candidate key in R: K1 $\cup$ K2
- Referential integrity constraints:
  R[K1] $\subseteq$ R1[K1] and R[K2] $\subseteq$ R2[K2]

Mapping associations (0..1 or 1 on both roles)



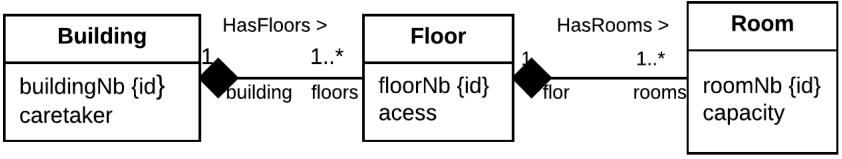
- 1—— 1. 2 options :
  1. K1 are attributes in R2 and R2[K1] = R1[K1],
     K1 is *another* candidate key in R2.
  2. K2 are attributes in R1 and R1[K2] = R2[K2],
     K2 is *another* candidate key in R1
- 1 —— 0..1. To avoid missing values, one option only:
  1. K1 are attributes in R2 and R2[K1] $\subseteq$ R1[K1],
     K1 is *another* candidate key in R2.

Association with Composition

We consider buildings located on a campus:

- Buildings have a caretaker and they are identified by their number.
- Building floors have specific access means, and in a given building, floors are identified by their number.
- Rooms have a fixed capacity, and in a given floor they are identified by their number.

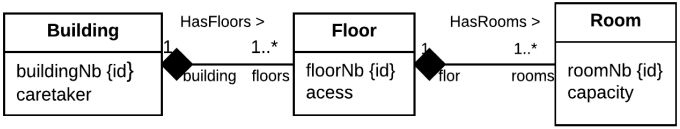
1. what are the **classes** involved?;
2. what are the **associations** involved ? (rel. name?, composition? multiplicity?, roles?)
3. what are the **identifiers** of the classes?



Association with Composition

For association with composition:

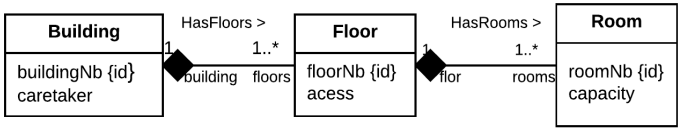
- The key of the 'whole' is included the key of the 'part'



Association with Composition

For association with composition:

- The key of the 'whole' is included the key of the 'part'



The relation schema is:

Buildings (buildingNb, caretaker)
Floors (floorNb, buildingNb, access)
Rooms (roomNb, floorNb, buildingNb, capacity)

Referential integrity constraints:
Floors[buildingNb] = Buildings[buildingNb]
Rooms[floorNb, buildingNb] = Floors[floorNb, buildingNb]

Not null contraints are implicitly associated with keys.

Question 6 : UML Composition to Relational (class B)

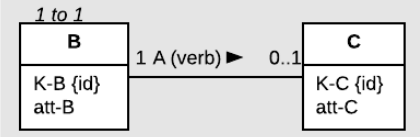


A. B(<u>K-B</u>, att-B)

B. B(<u>K-B, K-A</u>, att-B)

C. B(<u>K-B</u>, <u>K-A</u>, att-B)

Question 7 : UML 1 to 1 Association to Relational



A. B(<u>K-B</u>, att-B); C(<u>K-C</u>, att-C, <u>K-B</u>)
B. B(<u>K-B</u>, att-B); C(<u>K-C</u>, att-C, K-B)
C. B(<u>K-B</u>, att-B); A(<u>K-B,K-C</u>); C(<u>K-C</u>, att-C)

Mapping other associations except specialisation

- N-ary associations:
  As strongly advised by UML, map it first into classes and binary associations!

Chapter 4 – UML to REL

7 Introduction

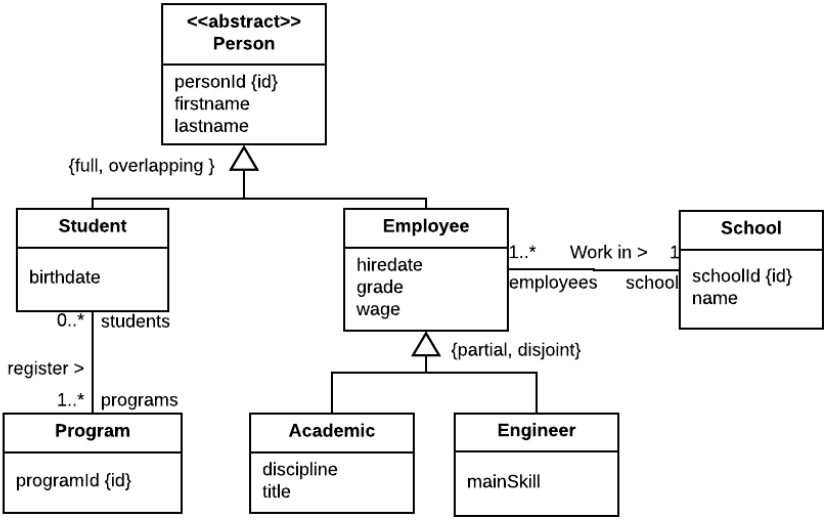
8 Mapping classes

9 Mapping Associations and Association classes

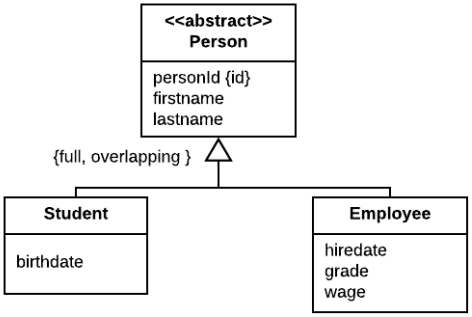
10 Mapping class Hierarchy

11 Concluding Remarks

Example: About a University

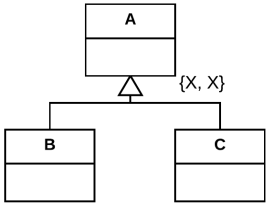


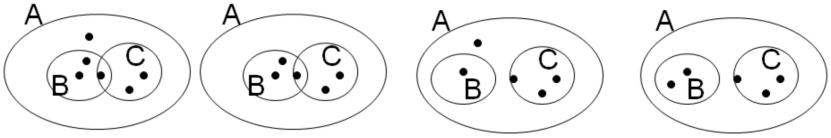
Example: About a University



- An employee is a person (with a first_name and last_name) who has a hiredate, a grade and a wage
- A student is a person who has a birthdate
- A person may only be an employee or a student (full)
- A person can be an employee, a student or both (overlapping)

Inheritance constraints

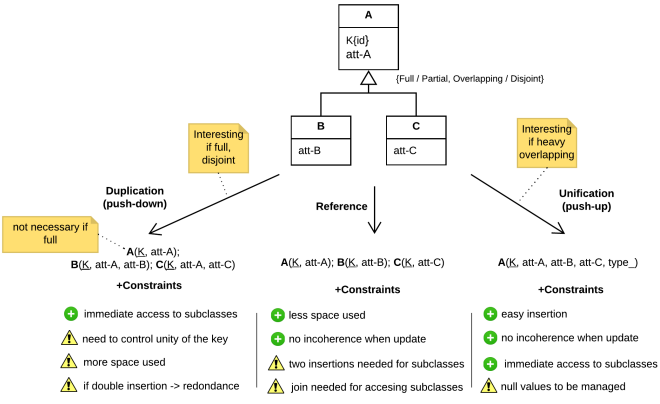




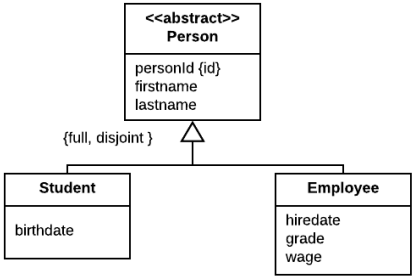
{partial, overlapping} {full, overlapping} {partial, disjoint} {full, disjoint}

Inheritance Mapping : three approaches

Usually, inheritance constraints may guide the used approach



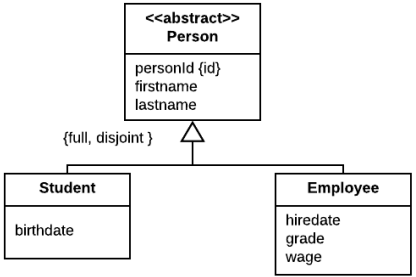
The University, Persons (Full, Disjoint Constraints)



- Given a person p: p is either a student or an employee (not both)
- Due to "full" constraint, there is no need for a relation Persons (there is no person that is neither student nor employee)
  1. Put Person properties in both relations Students and Employees (Duplicatation - Push down)

The University, Persons, Duplication Version

Relations related to sub-classes of Person all contain properties inherited from Persons:

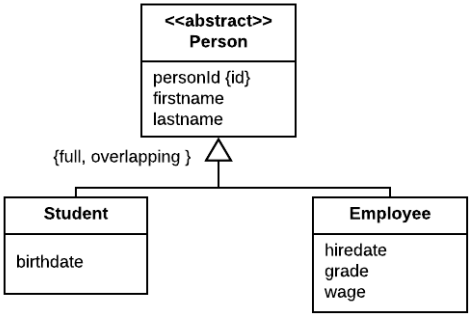


Students (personId, first_name, last_name, birthdate)
Employees (personId, first_name, last_name, hiredate, grade, wage)
Student[personId] $\cap$ Employee[personId] $= \emptyset$

$\pi_{\text{personId}}(\text{Students}) \cup \pi_{\text{personId}}(\text{Employees})$ returns all persons (view).

The University, Persons (Full, Overlapping Constraints)

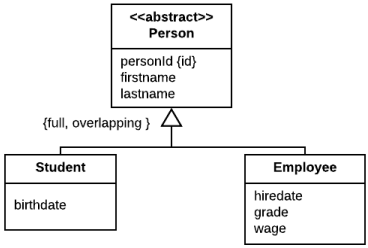
Focus on Person, Student and Employee with an "overlapping" constraint:



- Given a person p: p may be a student or an employee (or both)
- Define a relation Persons: attributes are those of the super-class
- Data related to a person who is student and employee are stored once only.

The University, Persons, Reference version

Focus on Person, Student and Employee with an "overlapping" constraint (we assume weak overlapping):

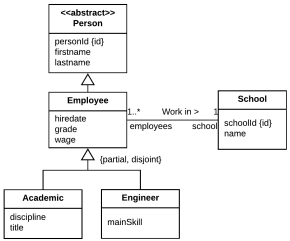


Persons (personId, first_name, last_name)
Students (personId, birthdate)
Employees (personId, hireDate, grade, wage)

$\text{Students[personId]} \subseteq \text{Persons[personId]}$
$\text{Employees[personId]} \subseteq \text{Persons[personId]}$

$\text{Students[personId]} \cup \text{Employees[personId]} = \text{Persons[personId]}$

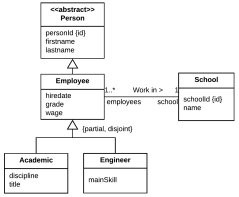
The University, Employees



As constraints are “partial” and “disjoint”:

- It may exist employees who are neither engineers nor academics. Thus, the relation Employees is necessary to store data about them.
- 2 options the relation Employees may represent:
  1. employees who are neither engineer nor academic (Duplication)
  2. all employees (Reference)

The University, Employees, Duplication version



Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
/ Contains only employees who are neither engineer nor academic. */*
Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)
/ Contains tuples for engineers only. */*
Academics (personId, hiredate, grade, wage, schoolId, title, discipline)
/ Contains tuples for academics only. */*

The University, Employees, Duplication version. Referencial Integrity Constraints

Employees represents employees who are neither engineer nor academic

Persons (personId, first_name, last_name)

Employees (personId, hiredate, grade, wage, schoolId)

/ Contains only employees who are neither engineer nor academic. */*

Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)

/ Contains tuples for engineers only. */*

Academics (personId, hiredate, grade, wage, schoolId, title, discipline)

/ Contains tuples for academics only. */*

Employees[personId] $\subseteq$ Persons[personId]

Engineers[personId] $\subseteq$ Persons[personId]

Academics[personId] $\subseteq$ Persons[personId]

Engineers[personId] $\cap$ Employees[personId] $= \emptyset$

Academic[personId] $\cap$ Employees[personId] $= \emptyset$

Academics[personId] $\cap$ Engineers[personId] $= \emptyset$

schoolId not null in Employees, Engineers and Academics.

Why this solution (Duplication)?

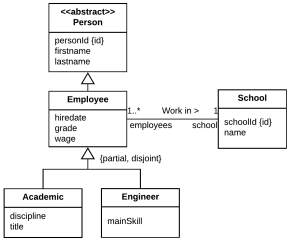
Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
/ Contains only employees who are neither engineer nor academic. */*
Engineers (personId, hiredate, grade, wage, schoolId, mainSkill)
/ Contains tuples for engineers only. */*
Academics (personId, hiredate, grade, wage, schoolId, title, discipline)
/ Contains tuples for academics only. */*

In this solution (Duplication),

- Accessing all details of employees according to her/his role is privileged.
- To retrieve all employees an union between Employees, Engineers and Academis must be performed (via a view).

The University, Employees, Reference version

Employee represents all employees:



Persons (personId, first_name, last_name)
Employees (personId, hiredate, grade, wage, schoolId)
/* *Contains one tuple for each employee.* */
Engineers (personId, mainSkill) /* *Engineers only.* */
Academics (personId, title, discipline) /* *Academics only.* */

The University, Employees, Reference version. Referential Integrity Constraints

Employee represents all employees

Employees (personId, hiredate, grade, wage, schoolId)
/ Contains one tuple for each employee. */*
Engineers (personId, mainSkill)
/ Engineers contains tuples for engineers only. */*
Academics (personId, title, discipline)
/ Academics contains tuples for academics only. */*
Employees[personId] $\subseteq$ Persons[personId]
Engineers[personId] $\subseteq$ Employees[personId]
Academics[personId] $\subseteq$ Employees[personId]
Engineers[personId] $\cap$ Academics[personId] $= \emptyset$
schoolId not null in Employees.

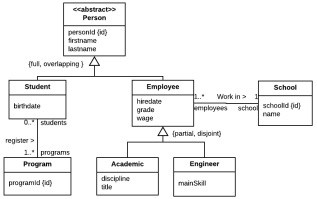
Why this solution (Reference)?

Employees (personId, hiredate, grade, wage, schoolId)
/ Contains one tuple for each employee. */*
Engineers (personId, mainSkill)
/ Engineers contains tuples for engineers only. */*
Academics (personId, title, discipline)
/ Academics contains tuples for academics only. */*

In this solution (Reference),

- Accessing all employees as a whole is privileged.
- Details of engineers (resp. academics) are retrieved with a natural join between relations Employees and Engineers (resp. Academics).

University: The global view of the transformation



+ Constraints defined before

Students (personId, first_name, last_name, birthdate)
Registrations (personId, programId);
Registrations[personId] = Students[personId] (1..*)
Registrations[programId] ⊆ Programs[programId] (0..*)
Programs (programId) -> Necessary because a Program can be unrealated with a student (0..*)
Schools (schoolId, name) Employees[schoolId] = Schools[schoolId]
Employees (personId, first_name, last_name, hiredate, grade, wage, schoolId)
Engineers (personId, mainSkill) *(Reference)* Engineers[personId] ⊆ Employees[personId]
Academics (personId, title, discipline) *(Reference)* Academics[personId] ⊆ Employees[personId]

Chapter 4 – UML to REL

7 Introduction

8 Mapping classes

9 Mapping Associations and Association classes

10 Mapping class Hierarchy

11 Concluding Remarks

- There is no theorem, nothing else than common sense!
- If something sounds wrong, go back to the conceptual design. Don't patch the relations!
- The relations quality is good: relations are at least in 3rd normal form ('good' relational model)
- Sometimes it needs a bit of clean up: check "useless" relations (included in an other one).